

PROVISIONAL PATENT APPLICATION

Title: Cross-Artifact Behavioral Coherence for Autonomous Agents Governed by Heterogeneous Persistent Context, Including Escalation-Windup Detection and Anti-Windup Reconciliation

Inventor(s): [to be completed] **Assignee:** [to be completed] **Status:** Draft for attorney review — engineering disclosure, not legal advice.

FIELD OF THE INVENTION

[0001] This disclosure relates to autonomous software agents whose behavior is governed by persistent context, and more particularly to detecting and reconciling contradictions that arise *across different classes* of persistent artifact — a typed memory store, a procedural skill, and a file or note — that are authored and updated independently and that carry different execution precedence at run time.

BACKGROUND

[0002] An autonomous agent typically maintains a memory store. Contemporary memory systems consolidate and reconcile that store offline: duplicates are merged, stale entries are refreshed, and contradictory entries are superseded to a latest value. Examples include background "dreaming"/consolidation passes, reflection over a memory stream, extract-and-update reconciliation pipelines, and temporally aware knowledge graphs that invalidate rather than delete. A related art records each write-side memory decision as a replayable, tamper-evident receipt so a memory state can be reconstructed, verified, and counterfactually re-evaluated.

[0003] A property common to all of the foregoing is that reconciliation operates **within the memory store**. The reconciler compares memory against memory. Two related approaches appear to reach outside the memory store but do not solve the present problem. First, a coding-agent memory that *validates a memory fact against a live code repository before use* and discards the fact when stale treats the non-memory artifact (the code) as implicit ground truth and always repairs the *memory*; it never attributes a contradiction to, or repairs, the non-memory artifact, and it does not model a non-memory artifact as a competing behavior-governing directive with its own execution precedence. Second, systems that *rank and deduplicate across plural memory channels* in a multi-agent setting combine multiple sources of the *same class* (memory streams) by relevance, recency, or trust to produce a better retrieval set; they do not span artifact classes of different execution character (a declarative memory versus a step-executed procedural skill), do not order artifacts by which one *governs behavior at execution time*, and emit a deduplicated retrieval list rather than a root-cause attribution and a repair of the governing artifact.

[0004] In practice, however, an agent does not learn from memory alone. Its effective behavior is governed by a *heterogeneous* set of persistent artifacts:

*a memory store of declarative facts/knowledge objects, updated continuously by feedback and dreaming, and consumed as advisory context; one or more **skills** — procedural instructions, frequently authored by the agent itself in a prior session — updated rarely and executed step-by-step, so that a skill tends to dominate behavior at run time; and *files**, notes, and working directories that the agent writes for itself and later references.*

These artifacts are authored on different cadences and carry different *execution precedence*. When they disagree, the agent’s actual behavior is determined by whichever artifact governs at execution time — frequently a skill — regardless of what the memory says.

[0005] This produces a failure mode that no memory-internal reconciler can resolve. A user repeatedly supplies the same corrective feedback. The feedback is admitted to memory, and because the corrective behavior never appears (a stale skill silently governs the behavior), the user repeats the feedback. The memory system, operating exactly as designed, *strengthens* the feedback memory on each cycle — raising its confidence, increasing its observation count, and tightening the directive — and yet behavior never changes. The strengthening continues without bound. Memory-versus-memory reconciliation finds no conflict, because the contradiction lives in a non-memory artifact the reconciler cannot see. The escalation is the only externally visible symptom of an invisible cross-artifact contradiction, and the system’s normally desirable property — faithfully strengthening a repeatedly observed signal — becomes a failure amplifier.

[0006] What is needed is a reconciliation layer that extends beyond the memory store: one that has situational awareness across all behavior-governing artifact classes, can detect a contradiction that spans those classes, can attribute the agent’s observed behavior to the artifact actually governing it, and can repair the governing artifact rather than producing yet another, stricter memory.

SUMMARY

[0007] Disclosed are systems, methods, and computer-readable media that maintain behavioral coherence of an autonomous agent governed by a plurality of persistent artifacts of different artifact classes. A coherence engine projects directives from each artifact class — a typed temporal memory store, a procedural skill artifact, and a file/note artifact — into a common directive representation having (a) a semantic identity computed from a topic embedding, (b) an artifact-class execution precedence, (c) a recency/staleness timestamp, and (d) for memory directives, an escalation-cycle count. The engine detects a cross-artifact contradiction between directives that share a semantic identity but originate in different artifact classes, attributes a governing artifact responsible for the agent’s behavior using execution precedence and staleness, and reconciles the contradiction by recording an auditable coherence decision that proposes a repair to the governing artifact, rather than (or in addition to) strengthening a memory directive.

[0008] In a first principal embodiment, the engine raises a **cross-artifact contradiction** incident when two directives sharing a semantic identity and originating in two different artifact classes assert mutually conflicting stances (for example, a memory directive that *requires* a behavior and a skill that *asserts* the behavior is already satisfied), and attributes the higher execution-precedence artifact as the governing artifact.

[0009] In a second principal embodiment — an **escalation-windup detector** — the engine determines that a memory directive has been monotonically strengthened across at least a threshold number of formation/reinforcement cycles, locates a non-memory artifact of higher execution precedence that governs the same semantic identity and whose last-updated time predates the escalation, and responsive thereto raises an incident that identifies the suspected non-memory governing artifact. The escalation count itself is used as the detector — the monotonic strengthening of a memory under repeated feedback is treated as the observable signature of a contradiction residing in a non-memory artifact. The engine applies an **anti-windup hold** to the escalating memory directive so that subsequent reinforcement records the repeated observation as evidence but no longer strengthens (freezes the confidence of) the directive, halting the runaway escalation while the governing artifact is repaired.

[0010] In embodiments, the projection reuses an embedding substrate also used for write-side semantic deduplication; the participating memory types and the per-class execution precedence are configured by a policy object; the named memory-formation policy object (a Motive) scopes which artifact classes participate; each coherence incident is recorded as a tamper-evident receipt; and an incident records that the governing artifact was self-authored by the agent in a prior session and predates the feedback. The coherence engine performs read and analysis without mutating raw evidence, and runs either on a schedule as a distinct dream-job kind or on demand as an operator action.

[0011] The disclosed systems provide a reconciliation that spans artifact classes rather than only the memory store; a root-cause attribution that names the artifact governing behavior rather than the memory being fruitlessly escalated; a control-theoretic anti-windup mechanism that converts the agent’s own escalation metric into a contradiction detector and then arrests the escalation; and an auditable decision trail that proposes a repair of the correct artifact.

BRIEF DESCRIPTION OF THE DRAWINGS

[0012] FIG. 1 is a block diagram of the cross-artifact coherence architecture: a memory store, registered skill/file artifact sources, a directive projector, a coherence directory, a contradiction/windup detector, a culprit attributor, and a reconciliation/decision recorder with an anti-windup hold.

[0013] FIG. 2 is a diagram of the common Directive representation and of the coherence directory keyed by directive semantic identity, showing per-artifact copies and their execution precedence and staleness.

[0014] FIG. 3 is a flowchart of the cross-artifact contradiction detection and culprit-attribution method.

[0015] FIG. 4 is a flowchart of the escalation-windup detector and the anti-windup hold actuator, including the freeze of directive strengthening on subsequent reinforcement.

[0016] FIG. 5 is a dataflow diagram of the feedback-futility escalation failure mode and its resolution: repeated feedback escalating a memory directive while a stale higher-precedence skill governs behavior, and the attribution of the skill as the culprit.

DETAILED DESCRIPTION

Architecture overview

[0017] Referring to FIG. 1, a memory store (100) holds typed temporal relationships, each carrying at least a subject, a predicate, an object, a memory type, a confidence, an observation count, a topic embedding, and recency timestamps. One or more artifact sources (105) provide non-memory persistent artifacts associated with a scope; an artifact source reads, for example, the agent’s live skill registry or a file/notes directory, and yields persistent artifact records each comprising an artifact identifier, an artifact class, an author, a self-authored flag, a last-updated timestamp, an optional location, and one or more declared directives. A directive projector (110) projects (i) active behavioral memory relationships of participating memory types and (ii) the declared directives of the registered artifacts into a common directive representation (200, FIG. 2). A coherence policy (115) supplies a semantic identity threshold, an escalation-cycle threshold, a per-artifact-class execution precedence mapping, the participating memory types, and detection toggles. A detector (120) evaluates cross-artifact contradiction (FIG. 3) and escalation windup (FIG. 4). A culprit attributor (125) selects, among contradicting directives, the artifact that governs behavior using

execution precedence and staleness. A reconciliation/decision recorder (130) emits an auditable coherence decision for each incident and, for a windup, applies an anti-windup hold (135) to the escalating memory directive in the memory store. The detection and analysis read the memory store and artifacts but do not mutate raw evidence.

[0018] Non-deterministic computation, if any, is confined to optional advisory components. The projection, the embedding-based identity, the precedence and staleness comparisons, the contradiction and windup tests, the attribution, and the hold are deterministic and auditable.

Definitions

[0019] An **artifact class** is one of a memory store, a procedural skill, or a file/note. A **directive** is a normalized representation of a behavior-governing instruction comprising a subject (a topic), an instruction, a stance, an artifact class, an artifact identifier, an execution precedence, a last-updated time, and, for a memory directive, an observation/escalation count and a confidence; a directive further carries a topic embedding used for semantic identity. A **stance** is a value from a closed vocabulary describing how a directive relates to a behavior on its subject, including *require*, *forbid*, *prefer*, *assert* (claims a state already holds), and *guide* (encodes a procedure). An **execution precedence** is an ordering over artifact classes reflecting which class governs behavior at run time; in a default embodiment a skill outranks a file, which outranks memory, because a skill is executed step-by-step while memory is advisory context. A **semantic identity** between two directives is a similarity (e.g. cosine of topic embeddings) at or above a configured threshold. **Staleness** of an artifact relative to a memory directive means the artifact's last-updated time precedes the memory directive's latest escalation time, or is unknown.

Common directive representation and the coherence directory

[0020] Referring to FIG. 2, the projector (110) maps each governing artifact into a Directive (200) regardless of class. A memory directive is projected from an active relationship of a participating memory type: its subject (topic) is taken from the relationship object text, its instruction from the relationship fact, its stance defaulted to *require* (memory directives being normative), its observation count from the relationship observation count, its confidence from the relationship confidence, its last-updated time from the relationship's most recent reinforcement (or last-seen, or valid-from) time, and its topic embedding reused from the relationship's stored object embedding. A skill or file directive is projected from a declared directive of a registered artifact, taking its subject, instruction, and stance as declared, its execution precedence from the policy mapping for the artifact's class, and its last-updated time and author from the artifact. The resulting directives are grouped by semantic identity into a coherence directory keyed by topic, each entry recording the artifacts that assert a copy of a directive on that topic together with their execution precedence and staleness.

[0020a] Projecting a non-memory artifact into one or more stanced directives is performed by one of two disclosed paths, so the projection is enabled for both structured and unstructured artifacts. In a declared-directive path, an artifact source supplies a manifest in which each directive's subject (topic), instruction, and stance are stated explicitly (for example, a structured front-matter block in a skill file or a sidecar descriptor); this is the deterministic, hermetic default and requires no language model. In an extracted-directive path, a directive extractor derives, from arbitrary artifact content (skill code, prose notes), one or more candidate directives by (i) producing a topic and an instruction span and (ii) classifying a stance into the closed stance vocabulary (*require*, *forbid*, *prefer*, *assert*, *guide*) — for example, a skill whose body asserts a pass condition ("a check passes when X") is classified as the *assert* stance, while a skill whose body prescribes a step is classified

as the guide stance. The extractor may reuse the same extraction transport and prompt-contract machinery used to extract memories from episodes, and the topic embedding is produced by the same embedding substrate used elsewhere for write-side semantic deduplication, so no additional model or storage contract is introduced. An artifact for which neither a declared directive nor an extractable directive is available is omitted from the coherence directory rather than guessed, failing safe.

Cross-artifact contradiction detection and attribution

[0021] Referring to FIG. 3, for each pair of directives originating in two different artifact classes whose topic embeddings have a cosine at or above the identity threshold and whose stances are mutually conflicting under a configured conflict relation, the engine raises a cross-artifact contradiction incident. In an embodiment the conflict relation includes the pairs *require/forbid*, *require/assert*, *forbid/guide*, and *forbid/assert*; the central case is *require* versus *assert*, in which a memory requires a behavior while a skill asserts the behavior is already satisfied. The culprit attributor (125) selects the directive of higher execution precedence as the governing directive and the other as the contended directive, and the incident records an attribution rationale and a proposed repair that reconciles the contended artifact against the governing artifact (or updates the governing artifact when the contended artifact is the intended truth).

Escalation-windup detection and anti-windup reconciliation

[0022] Referring to FIG. 4 and FIG. 5, the escalation-windup detector evaluates each memory directive whose observation/escalation count is at least the configured escalation-cycle threshold — i.e., a directive that has been reinforced across at least a threshold number of feedback cycles, the signature of the same feedback being supplied repeatedly. For such a memory directive the engine searches the projected non-memory directives for a candidate that (a) has a higher execution precedence than memory, (b) shares a semantic identity with the memory directive at or above the identity threshold, and (c) is stale relative to the memory directive (its last-updated time predates the latest escalation, or is unknown). Among qualifying candidates the engine selects, as the governing artifact, the candidate of highest execution precedence and then highest identity, and raises an escalation-windup incident. The incident records the escalating memory directive, the attributed governing artifact, the escalation count, the execution-precedence gap, the identity similarity, and an attribution rationale explaining that the repeated feedback is being absorbed into memory while the higher-precedence, staler artifact silently overrides it at execution time, so the memory escalates but behavior never changes. The incident records a proposed repair to update or quarantine the governing artifact and to suppress further blind escalation of the memory directive. In embodiments the incident further records that the governing artifact was self-authored by the agent in a prior session.

[0023] Responsive to a windup incident, and where permitted by policy, the engine applies an anti-windup hold to the escalating memory directive by marking the corresponding relationship with a hold indicator and the incident identifier. Thereafter, when the same feedback is observed again and would ordinarily reinforce the relationship, the formation pipeline records the new observation as evidence (extending observation count and evidence lineage) but does not strengthen the directive: its confidence is frozen at the held value rather than max-pooled upward, and a suppression count and timestamp are recorded. The hold is the control-theoretic anti-windup actuator: it arrests the runaway integration of the repeated error signal while the true contradiction — resident in the governing artifact — is repaired. The hold indicator is absent on ordinary relationships, so

the formation pipeline’s behavior is unchanged when no windup has been detected.

Decision recording, scheduling, and isolation

[0024] Each coherence incident is recorded as an auditable decision in the same audit log used for offline dream-agent decisions, the decision carrying a decision type identifying the incident kind, a summary, the contested subject, the scope, and a structured payload comprising the full incident (including the attributed governing artifact, the proposed repair, the escalation count, and the identity similarity). A query interface reconstructs incident objects from the persisted decisions, optionally filtered by scope and status, so incidents survive process restarts. The coherence engine runs either as a distinct dream-job kind on a configured cadence over a scope, or on demand as an operator action over a scope; in both cases it shares a single canonical analysis path. Artifact sources are scope-isolated: an artifact registered under one scope cannot raise an incident in another scope.

Policy and reuse

[0025] A coherence policy object configures the identity threshold, the escalation-cycle threshold, the per-artifact-class execution precedence, the participating memory types (by default the procedural/normative types such as directive and requirement), and independent toggles for the contradiction and windup detectors and for the anti-windup hold. In embodiments the named memory-formation policy object (a Motive) that governs autonomous writes also scopes which artifact classes participate in coherence and the precedence among them. The topic embedding reuses the embedding substrate used elsewhere for write-side semantic deduplication and clustering, so semantic identity across artifact classes requires no additional model. The identity threshold is configurable and calibrated to the embedding substrate in use; in a default embodiment using a hermetic character-trigram/token-unigram substrate, a threshold of approximately 0.50 cleanly separates same-topic directives (observed cosine ≈ 0.8) from unrelated directives (observed cosine ≈ 0.2), and a hosted sentence-embedding substrate would use a correspondingly higher threshold. Topic identity alone does not raise an incident: a windup incident additionally requires the conjunctive execution-precedence-and-staleness gate, and a contradiction incident additionally requires a conflicting stance across two different artifact classes, so the identity threshold governs candidacy rather than the incident, which substantially reduces spurious cross-artifact matches. The memory store may include a context-visible working tier and a full evidence tier; the coherence engine reads active directives from the working tier and records incidents and holds without disturbing the evidence tier.

Worked example

[0026] An agent evaluates the cost efficiency of other agents. A user repeatedly instructs the agent to be more thorough in that evaluation. The instruction is admitted as a memory directive of procedural type; across four feedback cycles its observation count rises to four while its confidence is held at a moderate value by max-pooling. A skill named "cost-efficiency-evaluator," authored by the agent in a prior session a month earlier and never updated, encodes a procedure that deems the evaluation satisfied under a static budget. Ordinary dreaming reconciles the memory against other memories and finds no conflict, so the directive continues to escalate. The coherence engine projects the four-times-reinforced memory directive (precedence 10) and the skill directive (precedence 30). It finds the skill shares the cost-efficiency topic (identity cosine $\approx 0.81 \geq 0.50$), has higher precedence (gap 20), and was last updated before the latest escalation. It raises an escalation-windup incident

attributing the stale self-authored skill as the governing artifact, records a decision proposing repair of the skill, and applies an anti-windup hold to the memory directive. A subsequent re-statement of the feedback at high confidence is recorded as evidence (observation count rises to five) but does not raise the directive’s confidence — the escalation is arrested, and the operator is directed at the actual cause.

Generalizations

[0027] The artifact classes may be extended to additional persistent behavior-governing artifacts (for example, configuration, tool definitions, or prompt templates), each assigned an execution precedence. The conflict relation among stances may be extended or learned. The semantic identity may use any embedding or similarity function. The staleness signal may incorporate version counters or content hashes in addition to timestamps. The anti-windup actuator may additionally gate retrieval of the held directive, route the incident to a human reviewer, or open a repair task against the governing artifact. In a separate embodiment, the coherence decision may be persisted using the replayable, tamper-evident receipt mechanism of the commonly-owned replay-receipts disclosure (incorporated herein by reference) so that a coherence analysis can be verified and re-evaluated; that receipt mechanism is not a limitation of the present claims, and the coherence incidents disclosed here are persisted in an auditable decision log without requiring hash-chained or Merkle-committed receipts.

CLAIMS

Invention I — Cross-artifact behavioral coherence

1. A computer-implemented method for maintaining behavioral coherence of an autonomous agent governed by a plurality of persistent artifacts of different artifact classes, comprising: projecting, into a common directive representation, (i) one or more memory directives from a memory store and (ii) one or more non-memory directives from a persistent artifact of an artifact class different from the memory store, each directive in the common representation having a semantic identity derived from a topic and an execution precedence associated with its artifact class; detecting a cross-artifact contradiction between a first directive projected from a first artifact class and a second directive projected from a second artifact class different from the first, the first and second directives sharing the semantic identity; attributing, as a governing artifact, the artifact whose directive has the higher execution precedence; and recording a coherence decision that identifies the governing artifact and a proposed repair of the governing artifact.

2. The method of claim 1, wherein the artifact classes comprise a typed temporal memory store, a procedural skill artifact that is executed by the agent, and a file or note artifact, and wherein the execution precedence ranks the skill artifact above the memory store.

3. The method of claim 1, wherein the semantic identity is determined from a cosine similarity between topic embeddings of the first and second directives at or above a configured identity threshold, and wherein the topic embeddings are produced by an embedding substrate also used for write-side semantic deduplication of the memory store.

4. The method of claim 1, wherein detecting the cross-artifact contradiction comprises determining that a stance of the first directive and a stance of the second directive are mutually conflicting under a configured conflict relation over a closed stance vocabulary, the stance vocabulary including a require stance, a forbid stance, and an assert stance that claims a behavior already holds.

5. The method of claim 4, wherein the first directive is a memory directive having the require stance and the second directive is a skill directive having the assert stance, whereby a memory requiring a behavior is reconciled against a skill asserting the behavior is already satisfied.

6. The method of claim 1, wherein the non-memory directive is projected from a persistent artifact record comprising an artifact identifier, an author, a last-updated time, and a self-authored flag indicating the artifact was authored by the agent in a prior session, and wherein the coherence decision records that the governing artifact is self-authored.

7. The method of claim 1, further comprising recording the coherence decision in an audit log as a structured payload from which a coherence incident object is reconstructable responsive to a query filtered by at least one of a scope and an incident status.

8. The method of claim 1, wherein projecting the memory directives comprises selecting active relationships of the memory store whose memory type is within a configured set of participating memory types comprising at least a procedural directive type and a requirement type.

9. The method of claim 1, wherein the persistent artifacts are scope-isolated such that a persistent artifact associated with a first scope is excluded from coherence detection for a second scope different from the first.

10. The method of claim 1, wherein a named memory-formation policy object that governs autonomous writes to the memory store further specifies which artifact classes participate in the projecting and the execution precedence among them.

11. The method of claim 1, wherein the projecting, detecting, attributing, and recording are performed by a distinct coherence dream-job kind executed on a configured cadence over a scope, the coherence dream-job kind being dispatched alongside formation, consolidation, and pruning dream-job kinds.

12. The method of claim 1, wherein the governing artifact is of an artifact class other than the memory store, and wherein the proposed repair modifies the governing artifact rather than the memory directive, whereby a contradiction is resolved by repairing a behavior-governing non-memory artifact instead of treating the memory as the artifact to be corrected.

13. The method of claim 1, wherein recording the coherence decision comprises emitting an output that identifies the governing artifact as a root cause of the agent's behavior, without modifying any artifact, whereby a detect-only embodiment surfaces the responsible artifact for review.

14. The method of claim 1, wherein the execution precedence is assigned to an individual artifact, in addition to or in place of being assigned per artifact class, the execution precedence reflecting which artifact governs the agent's behavior at execution time.

15. A system comprising one or more processors and memory storing instructions that, when executed, cause the system to perform the method of any of claims 1–14; and a non-transitory computer-readable medium storing instructions that, when executed, cause one or more processors to perform the method of any of claims 1–14.

Invention II — Escalation-windup detection and anti-windup reconciliation

16. A computer-implemented method for detecting and arresting a feedback-futility escalation in an autonomous agent, comprising: maintaining, in a memory store, a memory directive having an escalation count that increases each time a corresponding observation reinforces the memory directive; determining that the escalation count satisfies an escalation-cycle threshold; identifying a persistent artifact of an artifact class different from the memory store that (a) has an execution precedence higher than the memory store, (b) shares a semantic identity with the memory directive, and (c) has a last-updated time that predates a latest escalation of the memory directive; responsive

to said determining and identifying, raising an incident that identifies the persistent artifact as a suspected governing artifact of the agent’s behavior; and applying an anti-windup hold to the memory directive such that a subsequent observation reinforcing the memory directive is recorded as evidence without strengthening the memory directive.

17. The method of claim 16, wherein applying the anti-windup hold comprises marking the memory directive with a hold indicator, and wherein a subsequent reinforcement of the marked memory directive extends an observation count and an evidence lineage of the memory directive while freezing a confidence of the memory directive rather than increasing it.

18. The method of claim 16, wherein the escalation count is an observation count incremented by a write-side reinforcement that occurs when a later observation matches the memory directive, whereby a monotonic strengthening of the memory directive under repeated feedback is used as a detector of a contradiction residing in a non-memory artifact.

19. The method of claim 16, wherein, among a plurality of persistent artifacts satisfying conditions (a), (b), and (c), the suspected governing artifact is selected as the artifact having the highest execution precedence and thereafter the highest semantic identity to the memory directive.

20. The method of claim 16, wherein the incident records an escalation count, an execution-precedence gap between the suspected governing artifact and the memory store, a semantic-identity similarity, and an attribution rationale stating that the repeated feedback is being absorbed into memory while the suspected governing artifact overrides the behavior at execution time.

21. The method of claim 16, wherein the semantic identity is determined from a cosine similarity between a topic embedding of the memory directive and a topic embedding of a directive declared by the persistent artifact, the topic embedding of the memory directive being reused from an object embedding stored on the memory directive for write-side semantic deduplication.

22. The method of claim 16, further comprising recording the incident as a coherence decision in an audit log and, after the suspected governing artifact is repaired, clearing the anti-windup hold.

23. The method of claim 16, wherein the raising is further conditioned on a reconciliation internal to the memory store having found no contradiction for the memory directive, whereby a faithful monotonic strengthening of the memory directive — the memory subsystem operating as designed — is itself the failure signature that a contradiction resides outside the memory store.

24. The method of claim 16, wherein the escalation count comprises a monotonic reinforcement signal of the memory directive, and wherein condition (c) is satisfied when the persistent artifact is stale relative to the latest escalation of the memory directive or when the contradiction is unresolved by a reconciliation internal to the memory store.

25. The method of claim 16, wherein applying the anti-windup hold comprises attenuating or suspending further strengthening of the memory directive while continuing to record subsequent observations as evidence.

26. A system comprising one or more processors and memory storing instructions that, when executed, cause the system to perform the method of any of claims 16–25; and a non-transitory computer-readable medium storing instructions that, when executed, cause one or more processors to perform the method of any of claims 16–25.

ABSTRACT

An autonomous agent is governed not only by a memory store but by a heterogeneous set of persistent artifacts — memory, procedural skills, and files — that are authored on different cadences

and carry different execution precedence, so that a stale skill can silently override repeated feedback while memory-versus-memory reconciliation remains blind to the conflict. A coherence engine projects directives from every artifact class into a common representation having a topic-embedding semantic identity and a per-class execution precedence, detects contradictions that span artifact classes, and attributes the artifact that governs behavior by execution precedence and staleness. In a key embodiment, the monotonic strengthening of a memory directive across repeated feedback cycles is itself used as a detector: when a higher-precedence, staler non-memory artifact governs the same subject, the engine raises an escalation-windup incident identifying the suspected governing artifact and applies an anti-windup hold that records further feedback as evidence without strengthening the memory directive, arresting the runaway escalation while the correct artifact is repaired. Each incident is recorded as an auditable decision proposing a repair of the governing artifact rather than yet another stricter memory.

FIG. 1

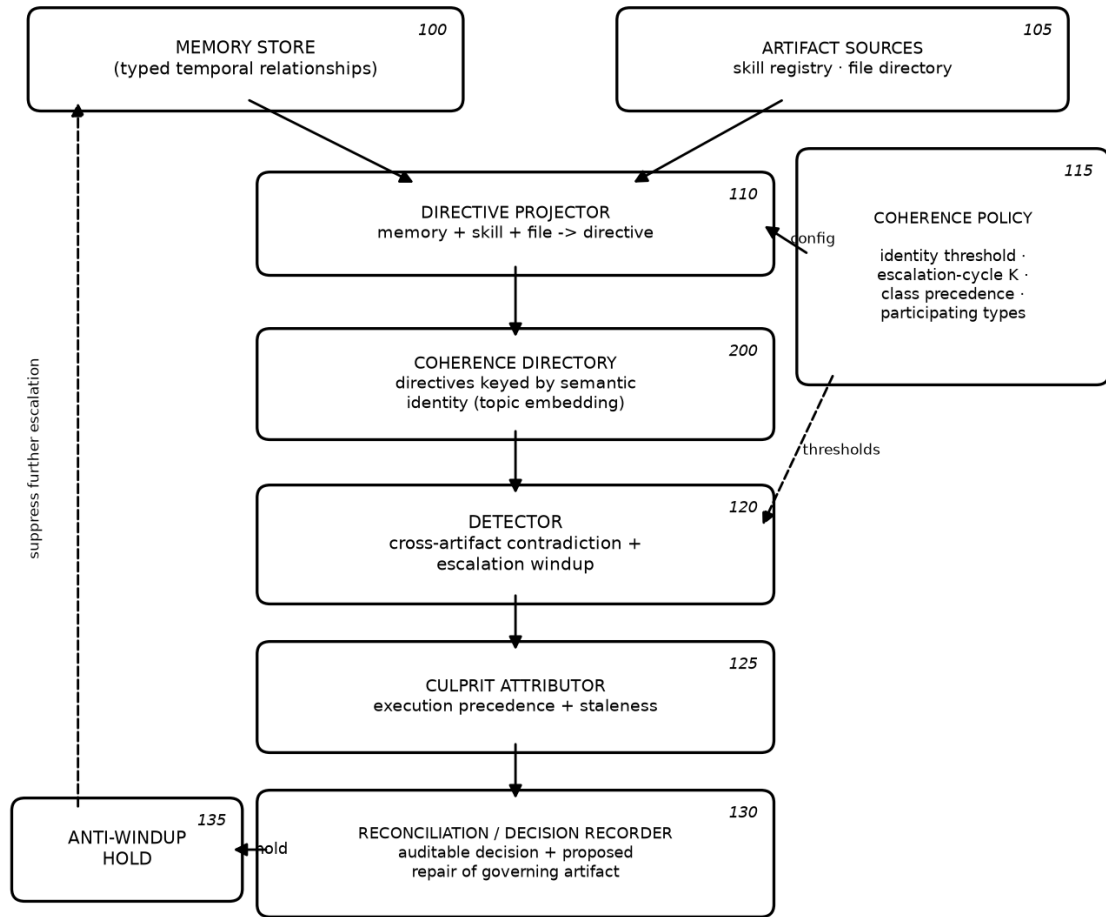


FIG. 1 — Cross-artifact coherence architecture.

FIG. 2

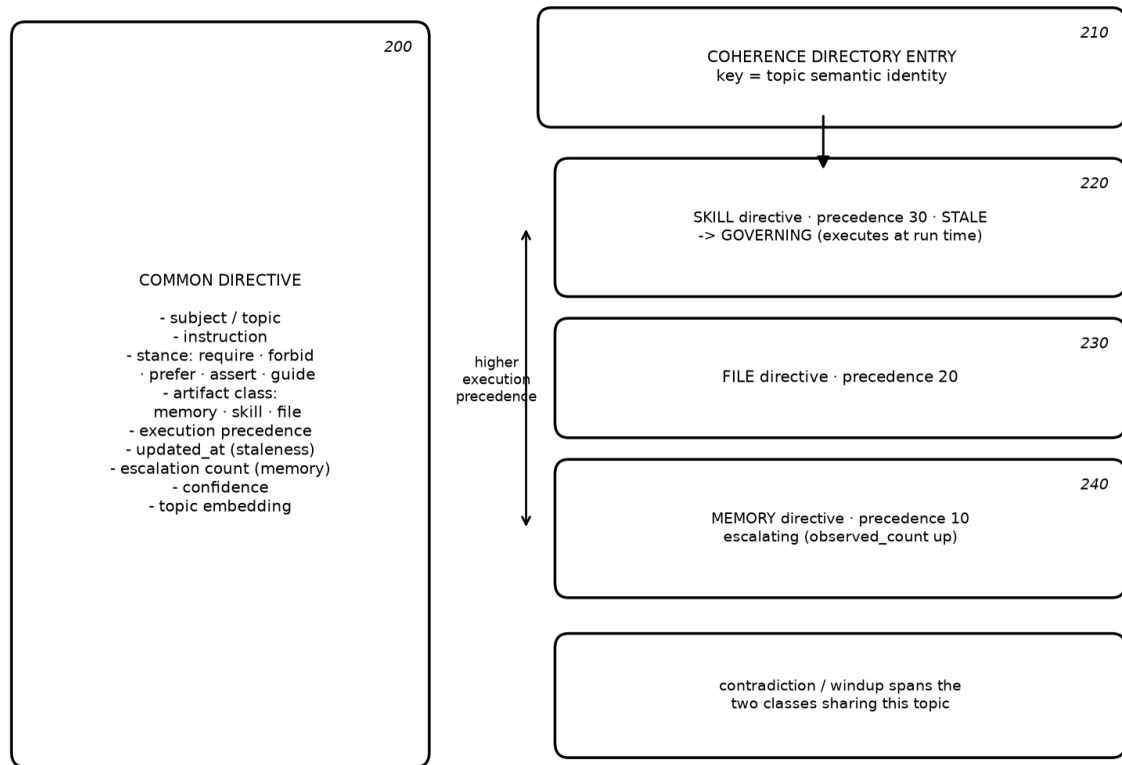


FIG. 2 — Common directive representation and the coherence directory keyed by semantic identity.

FIG. 3

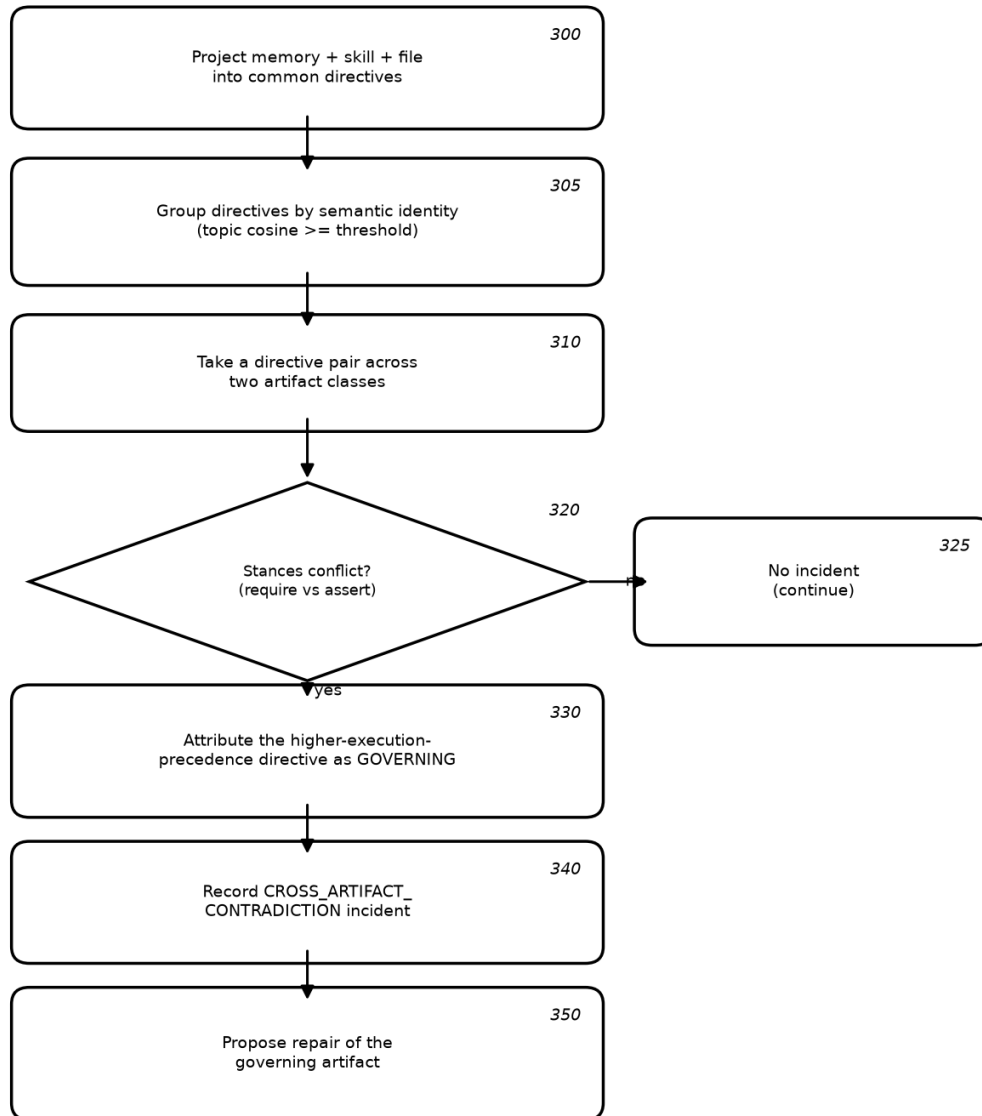


FIG. 3 — Cross-artifact contradiction detection and culprit attribution.

FIG. 4

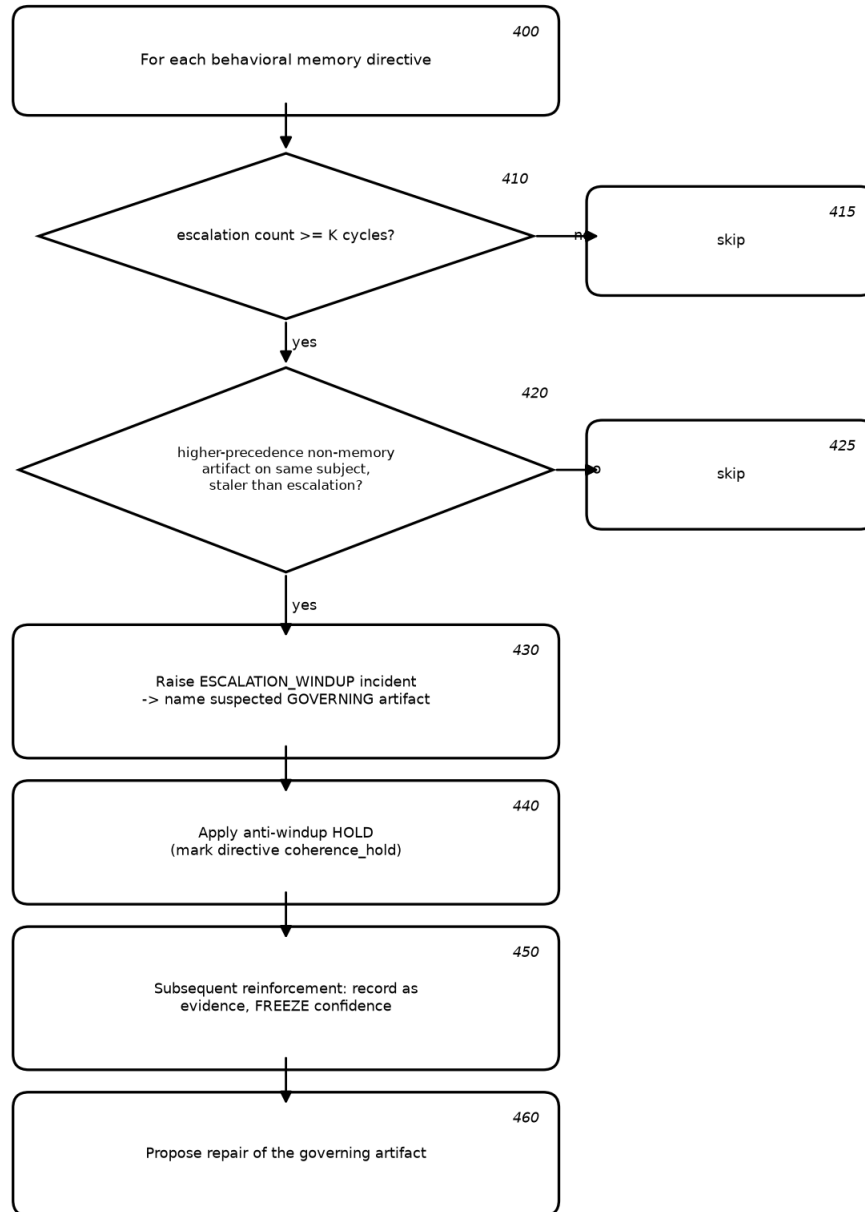


FIG. 4 — Escalation-windup detector and anti-windup hold actuator.

FIG. 5

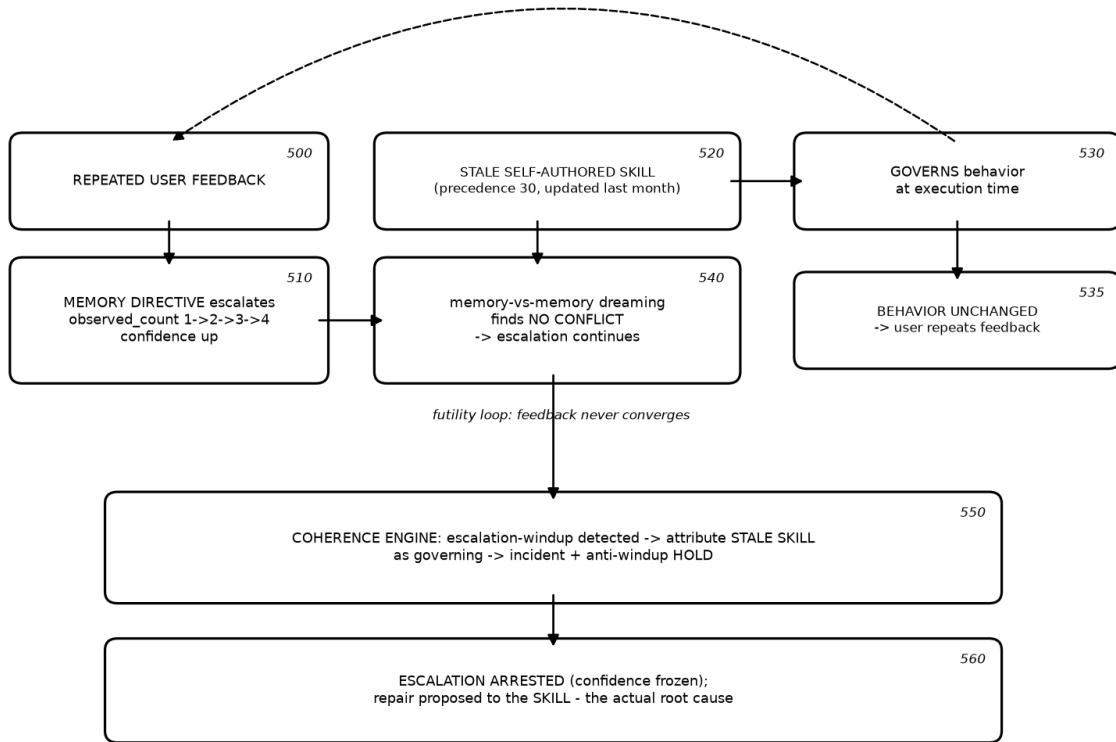


FIG. 5 — Feedback-futility escalation failure mode and its resolution.